QA

Portfolio Case Study
Web application quality assurance

E-COMMERCE · AI-ASSISTED ORDERING · ROLE-BASED OPERATIONS

AI-CAKE Quality Assurance Testing Portfolio

A practical QA case study covering customer ordering, staff operations, administrator workflows, authentication, capacity controls, and payment handoff validation.

System under test	Aure Sanchez House of Cakes / AI-CAKE
Test focus	Functional, regression, usability, authorization, deployment readiness
Test environment	PHP, MySQL, JavaScript, XAMPP local environment
Report date	05 October 2026

01 · QA OVERVIEW

Project context and QA contribution

AI-CAKE is a pickup-only cake-ordering system with normal cake ordering, AI-generated custom cake designs, staff-assisted ordering, administrator controls, SMS OTP verification, inventory and capacity checks, recommendations, and Xendit GCash handoff.

181

PHP files syntax-checked

Active application files passed PHP lint validation.

3

Role workspaces audited

Customer, staff, and administrator navigation and workflows.

2

Payment paths reviewed

Cash checkout and Xendit GCash handoff.

QA approach: defect discovery, root-cause isolation, route and redirect validation, role-based regression checks, data-flow review, and deployment-readiness assessment. Findings were recorded only when supported by source inspection or local smoke-check evidence.

Technology scope

PHP · MySQL · HTML/CSS · JavaScript · Bootstrap · cURL · Semaphore SMS · Stability AI · Xendit GCash

02 · DEFECT VERIFICATION

Verified defects and implemented corrections

ID	DEFECT / USER IMPACT	ROOT CAUSE	VALIDATION AND CORRECTION	STATUS
QA-01	Customer ordering navigation could lead away from the authenticated customer workspace.	Public storefront/menu and customer workspace navigation were mixed.	Added an authenticated customer menu destination and routed customer “Order a cake” actions to it. Public visitors retain the storefront flow.	**Fixed & verified**
QA-02	Staff Pickup Schedule order links did not reliably open the selected order in Unified Order Management.	The schedule link did not preserve the target order number in the receiving search filter.	Passed the selected order number through the Unified Order Management search query while retaining schedule filters.	**Fixed & verified**
QA-03	Administrator account action buttons could overflow the account-card layout on desktop and smaller widths.	Actions were constrained by an overly narrow grid column.	Reworked the account editor action area into a responsive layout while preserving account update/deactivation behavior.	**Fixed & verified**
QA-04	Customer and staff chat views lacked clear message hierarchy and the customer conversation entry was text-heavy.	Inconsistent conversation layout and weak visual distinction between sender roles.	Added an accessible icon entry point and consistent purple/lavender message-bubble presentation without changing ownership, CSRF, polling, or send logic.	**Fixed & verified**
QA-05	Separate legacy admin/staff entry pages conflicted with the required unified role-based login experience.	Legacy routes remained reachable alongside unified authentication.	Unified login routes users by database role; legacy entry pages redirect to the shared login route.	**Fixed & verified**
QA-06	Customer cart data could persist across users or after guest sign-in.	Browser storage needed account-specific isolation and controlled guest-cart merge behavior.	Validated per-user cart keys and one-time guest-to-customer merge behavior to prevent cross-account cart visibility.	**Fixed & verified**

Defect descriptions are written at a portfolio level and exclude API keys, customer data, OTP values, and private infrastructure details.

Regression areas tested

Customer journey

- Separate registration, OTP verification, login, and role routing.
- Authenticated catalog navigation, product customization, cart isolation, and checkout entry.
- Pickup date/time availability, stock checks, order history, receipts, cancellation rules, and favorites.
- AI design ownership, scheduling, staff pricing, design conversation, and order tracking.

Administrator journey

- Account creation, activation/deactivation, staff management, catalog, options, capacity, and reports.
- Responsive account actions and non-destructive inventory management paths.
- Administrator links into shared staff operational pages verified against role authorization.

Staff journey

- Unified normal/AI order queue, filters, price assignment, discounts, status sequencing, cancellation, and claim-code completion.
- Pickup Schedule filters and selected-order drill-down.
- Assisted cash order creation using the same capacity, inventory, and pricing rules as customer checkout.

Technical validation

- PHP syntax validation across 181 active PHP files.
- GET-only route smoke checks for public, customer, staff, and admin paths.
- Expected unauthenticated redirect behavior to unified login.
- Role revalidation, customer ownership, CSRF-protected mutation routes, and parameterized data access reviewed.

04 · RISK-BASED QA FINDINGS

Open risks identified for the next test cycle

Professional QA includes reporting unresolved risks separately from verified fixes. The following were identified during the latest audit and are not represented as resolved.

High priority — GCash cart and duplicate-order handling. The GCash browser branch redirects to Xendit before clearing the customer’s local cart. Repeated clicks can also create duplicate pending orders, and invoice-creation failure can leave stock/capacity reservations held until the expiry job runs.

High priority — production payment configuration. Hostinger release requires a live HTTPS application URL, business-owned Xendit production credentials, webhook callback token, verified callback endpoint, and Hostinger Cron Jobs for notifications/reminders/payment expiry.

Deployment hardening. Legacy backup folders must not be included in the public upload package. The server must preserve `.htaccess` protections and provide required PHP extensions plus writable upload storage.

QA recommendation

Run a controlled staging regression after the payment-flow fixes: one cash order, one Xendit test payment, one AI order, one cancellation/restoration test, staff completion with claim code, and a verified payment webhook callback. Record expected/actual results and attach screenshots for final release sign-off.

05 · PROFESSIONAL SUMMARY

QA value demonstrated

Defect analysis

Translated user-reported behavior into reproducible root causes across navigation, account UI, checkout, staff scheduling, and chat workflows.

Risk-based testing

Distinguished fixed defects from open payment and deployment risks rather than overstating release readiness.

Cross-role coverage

Validated end-to-end behavior across customer, staff, and administrator experiences.

Evidence-driven reporting

Used source review, syntax checks, route smoke tests, and controlled environment validation to support findings.

Portfolio statement: This case study demonstrates practical manual and technical QA work on a real PHP/MySQL business system. It reflects verified implementation and testing activity in a local development environment; it does not claim a completed public-production security certification.